

Keywords: diffusion language models • masked decoding • reasoning order • test-time inference

Answer First, Reason Later: Commitment Order in Diffusion LLMs

Unconstrained Masked Decoding Commits Final Answers Before Reasoning Completes, Collapsing to Answer-Only Outputs on Up to 90% of Problems

By Jewon Yeom, Jaewon Sok, Seonghyeon Park, Jeongjae Park, Hwiyeong Lee, Taesup Kim

arXiv:2608.05687 • Preprint, August 2026

Masked diffusion language models promise order-free generation as a core advantage over autoregressive decoding. A new study from Seoul National University shatters that assumption for reasoning tasks: logging every token commitment in LLaDA-8B on GSM8K reveals that unconstrained decoding commits the final answer at only 15–24% of the trajectory, before half the reasoning region is written, collapsing to answer-only outputs on up to 90% of problems as canvas size grows.

AT A GLANCE

Problem: Diffusion LLMs generate tokens in arbitrary order, but reasoning tasks require intermediate steps to precede the final answer — an ordering constraint the model violates.

Why it matters: dLLMs are increasingly proposed as reasoning engines; if their decoding procedure systematically skips reasoning, the architecture is unreliable for this use case.

Method: Log every token commitment (masked $\rightarrow$ unmasked transition) during LLaDA-8B decoding on GSM8K; analyze commitment position as a fraction of the decoding trajectory.

Result: Pure decoding commits the answer at 15–24% of the trajectory; constrained ordering recovers substantial accuracy without retraining.

Evidence: Collapse to answer-only outputs reaches 90% of problems at large canvas size; commitment trajectory visualizations confirm answer-first ordering.

The Big Picture

Masked diffusion language models (dLLMs) generate text by starting from a fully masked token sequence and iteratively unmasking tokens over multiple diffusion steps. Unlike autoregressive (AR) models, which are constrained

to generate left-to-right, dLLMs may unmask any position in any order. Proponents argue this flexibility enables superior global coherence and parallel generation.

For reasoning tasks, however, this flexibility has a fatal cost. A reasoning trace has inherent logical structure: the intermediate steps should be generated *before* the final answer, because the answer should be justified by preceding computation, not the reverse. When a dLLM commits the answer token before writing the reasoning steps, it is, in effect, guessing — and the reasoning region becomes post-hoc decoration for a decision already made.

Why Existing Approaches Fall Short

Standard dLLM decoding (“pure” diffusion) selects which tokens to unmask at each step based on model confidence, without any constraint on the semantic identity of the tokens being committed. The model may unmask the final answer position early because the answer is high-confidence given the problem statement alone — before the chain-of-thought region provides any additional information.

Prior work on improving dLLM reasoning has focused on training objectives (RLVR, supervised fine-tuning on CoT data) or on post-hoc verification. None has examined the *order* in which tokens are committed during decoding, which this paper identifies as the proximate cause of failure.

Core Mechanism

The authors introduce **commitment order logging**: at each diffusion step, they record which output position transitions from masked to unmasked, forming a complete commitment trajectory. For a generation with T positions and S decoding steps:

- Each step s produces a set $C_s \subseteq [T]$ of newly unmasked positions.
- The answer region $\mathcal{A} \subset [T]$ and reasoning region $\mathcal{R} \subset [T]$ are identified by position in the output template.
- The “answer commitment step” $s^* = \min\{s : C_s \cap \mathcal{A} \neq \emptyset\}$ is the fraction s^*/S of the trajectory at which the final answer is first committed.

The **constrained decoding** fix enforces that positions in $\mathcal{A}$ remain masked until a threshold fraction τ of the trajectory has elapsed, releasing the answer region only after substantial reasoning has been committed.

Technical Formulation

Let $\mathbf{x}_0$ be the target output and $\mathbf{x}_s$ the partially masked sequence at step s , with $\mathbf{x}_S$ fully masked. Pure reverse

diffusion selects tokens to unmask according to:

$$p_{\theta}(\mathbf{x}_{s-1} \mid \mathbf{x}_s) \propto \prod_{i \in [T]} p_{\theta}(x_i \mid \mathbf{x}_s)^{\mathbb{1}_{[x_s^{(i)} = \text{[MASK]}]}}$$

Commitment at step s selects the top- k unmasked positions by $p_{\theta}(x_i \mid \mathbf{x}_s)$.

Constrained decoding modifies the selection by adding a region-based gate:

$$p_{\theta}^{\text{con}}(x_i \mid \mathbf{x}_s) = p_{\theta}(x_i \mid \mathbf{x}_s) \cdot \mathbb{1}_{\left[i \notin \mathcal{A} \text{ or } \frac{s}{S} > \tau\right]}$$

where $\tau \in (0, 1)$ is the threshold fraction. No change to model weights is required; the constraint is purely at inference time.

Learning or Inference Procedure

No training is involved. Both pure decoding and constrained decoding are evaluated at inference time on LLaDA-8B, a masked diffusion language model with 8 billion parameters. GSM8K provides 1,319 test-set math word problems as the evaluation benchmark. The commitment threshold τ is tuned on a small held-out validation set.

What the Guarantee Says

No formal guarantee is provided. The empirical claim is that answer-first commitment is the dominant failure mode of pure dLLM decoding on reasoning tasks, and that constraining the answer region during early decoding consistently recovers accuracy. The correlation between canvas size and collapse rate (peaking at 90% answer-only outputs) establishes that the failure is deterministic as generation length grows, not stochastic.

Experimental Findings

- **Commitment timing:** Pure decoding commits the answer at 15–24% of the decoding trajectory on average.
- **Collapse rate:** Up to 90% of problems produce answer-only outputs (zero reasoning steps) as canvas size grows.
- **Constrained decoding:** Enforcing reasoning-before-answer commitment recovers substantial GSM8K accuracy with threshold $\tau = 0.5$.
- **Scale sensitivity:** The gap between constrained and unconstrained decoding widens with output length, confirming scale-dependent failure.

Ablations and Interpretation

Why does pure decoding answer first? The answer token has high local confidence given the problem

statement alone (the model “knows” plausible answer formats). The reasoning region is globally coherent only *after* intermediate steps are committed — but those steps are low-confidence before being written, so the decoding procedure deprioritizes them.

Why does collapse worsen with canvas size? Longer outputs have more reasoning positions that must be committed before the answer is released. As the reasoning region grows, the denominator of the commitment problem grows — making early, spurious answer commitment increasingly likely under unconstrained selection.

Threshold sensitivity: Accuracy peaks around $\tau = 0.4$ – 0.6 with a shallow optimum; extreme values ($\tau \rightarrow 0$ or $\tau \rightarrow 1$) recover the failure modes of pure decoding or fully blocked answer generation respectively.

Reference

Jewon Yeom, Jaewon Sok, Seonghyeon Park, Jeongjae Park, Hwiyeong Lee, Taesup Kim. **Answer First, Reason Later: Commitment Order in Diffusion LLMs.** arXiv:2608.05687, August 2026. <https://arxiv.org/abs/2608.05687>

Keywords: test-time scaling • retrieval-augmented reasoning
• large reasoning models • chain-of-thought

ThinkRetrieve: Retrieval-Augmented Reasoning Traces for Test-Time Scaling

Dynamically Injecting Solved Exemplars Mid-Reasoning Counters Error Compounding and Improves Accuracy Across Five Models and Four Benchmarks

By Vaibhav Singh, Soumya Suvra Ghosal, Sarvesh Gharat, Soumyabrata Pal, Ramasuri Narayanam, Dinesh Manocha

arXiv:2608.10928 • Preprint, August 2026 • IIT Bombay; Univ. of Maryland; Adobe Research

Sequential test-time scaling of Large Reasoning Models (LRMs) often yields diminishing returns: longer reasoning traces compound errors and drift from the original problem. ThinkRetrieve fixes this by dynamically retrieving solved step-by-step examples from an external corpus at each intermediate reasoning step, injecting guidance on *how* to reason rather than *what facts* are relevant. Experiments across five models (1.5B–8B parameters) on GSM-8K, MATH-500, AIME 2025, and SciQ show consistent accuracy gains over standard test-time scaling.

AT A GLANCE

Problem: Allocating more inference compute (longer CoT traces) often backfires due to error compounding and problem drift in Large Reasoning Models.

Why it matters: Test-time scaling is the dominant paradigm for improving LRM accuracy; fixing its failure mode is high-leverage and training-free.

Method: At each reasoning step, retrieve the top- m solved exemplars from an external corpus (conditioned on the partial trace) and inject them into the thinking trace.

Result: Consistent accuracy gains on GSM-8K, MATH-500, AIME 2025, and SciQ across five models; largest gains on hard benchmarks and small models.

Evidence: All five models (1.5B–8B) improve; gains replicated across four benchmarks spanning math and science reasoning.

The Big Picture

Large Reasoning Models (LRMs) extend the chain-of-thought paradigm to its logical limit: instead of a few reasoning steps, they generate extended thinking traces of hundreds or thousands of tokens before producing a final

answer. The more inference compute allocated, the better the performance — in principle. In practice, sequential test-time scaling saturates and then degrades: beyond some trace length, accuracy falls because the model loses track of the problem, makes errors it cannot catch, and compounds small mistakes into large final errors.

ThinkRetrieve reframes test-time scaling as a *guided* process. Rather than letting the model reason in isolation, it provides the model with solved analogues — problems whose solutions are structurally similar to the current reasoning step — at each point where the trace could diverge. The key insight is that retrieval is most useful when it is *dynamic*: the right exemplar at step 3 of a reasoning trace is different from the right exemplar at step 10.

Why Existing Approaches Fall Short

Standard RAG retrieves documents before generation and injects them once into the context. For reasoning tasks, this static approach misses the key failure mode: errors do not arise from factual ignorance but from *procedural* mistakes — applying the wrong operation, misidentifying the relevant sub-problem, or making an algebraic slip mid-trace. A retrieved document about a related topic does not provide procedural guidance.

Best-of- N sampling addresses error compounding by generating N independent traces and selecting the best one, but is expensive (linear in N) and does not actually fix any individual trace — it relies on at least one trace avoiding the failure mode. ThinkRetrieve fixes the trace *at the point of failure*.

Core Mechanism

ThinkRetrieve wraps the LRM’s generation loop with a retrieval step at each reasoning checkpoint:

1. At reasoning step k , form a query $q_k = [\text{problem}; \text{partial trace}_k]$.
2. Retrieve top- m solved exemplars (p_j, sol_j) from corpus $\mathcal{C}$ based on embedding similarity to q_k .
3. Inject the exemplars as additional in-context examples preceding step k .
4. Continue the LRM’s generation with the augmented context.

The corpus $\mathcal{C}$ is a collection of (problem, step-by-step solution) pairs; no fine-tuning of the LRM is performed. The retrieval encoder is a standard dense retriever (e.g., a contrastively trained bi-encoder).

Technical Formulation

Let π_θ be the LRM policy and x the input problem. Standard decoding generates the trace $t = (t_1, \dots, t_L)$ autore-

gressively:

$$p(t \mid x) = \prod_{k=1}^L p_{\theta}(t_k \mid x, t_{<k})$$

ThinkRetrieve modifies the conditioning at each step by inserting retrieved exemplars $\mathcal{E}_k = \text{Retrieve}(\mathcal{C}, [x; t_{<k}])$:

$$p^{\text{TR}}(t_k \mid x, t_{<k}) = p_{\theta}(t_k \mid \mathcal{E}_k, x, t_{<k})$$

where $\mathcal{E}_k$ is updated at each reasoning checkpoint (every δ tokens). The final answer is extracted from t_L as in standard decoding.

Retrieval similarity is cosine distance in the embedding space:

$$\text{sim}(q_k, p_j) = \frac{\text{enc}(q_k)^{\top} \text{enc}(p_j)}{\|\text{enc}(q_k)\| \cdot \|\text{enc}(p_j)\|}$$

Top- m exemplars by similarity are selected and prepended to the context.

Learning or Inference Procedure

No training is involved. The LRM (π_{θ}) and dense retriever are both frozen. The only hyperparameters are:

- m : number of retrieved exemplars per step (ablated: 1, 3, 5)
- δ : retrieval frequency (every δ generated tokens)
- Retrieval encoder: the paper uses a standard off-the-shelf bi-encoder

What the Guarantee Says

No formal guarantee is provided. The empirical claim is that dynamic step-conditioned retrieval consistently improves accuracy over static retrieval and over no retrieval, across model scales from 1.5B to 8B parameters and across four diverse benchmarks. The consistency across scales and tasks provides strong evidence that the mechanism is general rather than benchmark-specific.

Experimental Findings

- **GSM-8K and MATH-500**: Consistent accuracy gains across all five models; gains increase with trace length (longer traces benefit most).
- **AIME 2025**: Largest absolute gains, where baseline traces are most error-prone and problem drift is most severe.
- **SciQ**: Gains confirmed on a non-math reasoning benchmark, establishing cross-domain generality.
- **Small models (1.5B–3B)**: Largest relative gains; ThinkRetrieve provides the most leverage where baseline reasoning is weakest.

- **Ablation on m** : $m = 3$ is near-optimal; $m = 1$ is 60–80% of the gain, $m > 5$ provides marginal additional benefit at higher context cost.

Ablations and Interpretation

Dynamic vs. static retrieval: Retrieving once at the start (standard RAG) provides $\sim 30\%$ of the accuracy gain of dynamic per-step retrieval — confirming that the value is in the conditioning on the *partial trace*, not just the problem.

Retrieval timing: Injecting exemplars early in the trace (step 1–2) is most helpful for math benchmarks, where problem setup is the hardest phase; mid-trace retrieval dominates for multi-step science problems.

Corpus quality: Using a corpus of correct solutions consistently outperforms a corpus of incorrect-but-plausible solutions, confirming that the model is learning procedural structure rather than just surface similarity.

Reference

Vaibhav Singh, Soumya Suvra Ghosal, Sarvesh Gharat, Soumyabrata Pal, Ramasuri Narayanam, Dinesh Manocha. **ThinkRetrieve: Retrieval-Augmented Reasoning Traces for Test-Time Scaling**. arXiv:2608.10928, August 2026. <https://arxiv.org/abs/2608.10928>

Keywords: RLHF fairness • preference heterogeneity • reward modeling • alignment

Procedural Fairness Failures in RLHF from Preference Averaging

Standard RLHF Violates Procedural Fairness by Drowning Minority Preference Signals in Aggregated Reward Models; PA-RLHF Separates Optimization Across Preference Modes

By M P V S Gopinadh, Karthik Kamuju, Kummari Avinash, John Joshua, Srinivasa Raju Rudraraju

arXiv:2608.10126 • ICLR 2026 Workshop on Algorithmic Fairness Across Alignment Procedures (AFAA)

Reinforcement Learning from Human Feedback (RLHF) aggregates heterogeneous annotator preferences into a single reward model, treating all annotators as draws from one distribution. A new paper shows this induces a *procedural fairness failure*: majority preference groups dominate reward learning while minority preferences are systematically under-represented. Preference-Aware RLHF (PA-RLHF) remedies this by separating reward optimization across preference modes at the reward learning stage.

AT A GLANCE

Problem: Standard RLHF averages heterogeneous preferences into a single reward model, causing minority preference groups to be under-represented.

Why it matters: As LLMs are deployed across diverse populations, preference homogeneity is a poor assumption — and its failure is invisible to standard metrics.

Method: Define procedural fairness for alignment; prove standard RLHF violates it; propose PA-RLHF with per-group reward models.

Result: PA-RLHF improves procedural fairness metrics while maintaining outcome quality (win rates, helpfulness) across all tested scenarios.

Evidence: Gains largest when inter-group preference divergence and population size imbalance are greatest.

The Big Picture

Human preferences are not homogeneous. Annotators employed in RLHF pipelines come from different cultural backgrounds, hold different values, and have different conceptions of what constitutes helpful, harmless, and honest AI behavior. Standard RLHF treats all annotator preferences as if they were noisy draws from a single underlying distribution, fitting a single aggregated reward model r_θ to the pooled data.

When preferences are genuinely multi-modal — when subgroups of annotators hold coherently different values — this aggregation is not merely imprecise. It actively *dilutes* minority preference signals in proportion to the size of the minority. Smaller groups contribute less to the aggregate loss, so the reward model learns to satisfy the majority and discard the minority. The paper calls this a **procedural fairness failure**: the process is unfair to minority groups, independent of whether the resulting policy happens to satisfy them by coincidence.

Why Existing Approaches Fall Short

Prior work on fairness in RLHF has focused on **outcome fairness**: ensuring the final policy produces acceptable outputs for all subgroups. But outcome fairness can be achieved by coincidence (a policy that satisfies the majority may also happen to satisfy the minority on some tasks) and can be gamed by optimizing for population-level statistics. Outcome fairness does not address whether the reward learning *process* treats all groups equitably.

Existing personalized preference methods (per-user reward models, preference embeddings) require per-user data at inference time and scale poorly. PA-RLHF operates at the group level during reward training, requiring no per-user data after deployment.

Core Mechanism

Let $\mathcal{A}$ be the annotator pool, partitioned into K preference groups $G_1, \dots, G_K$ with group-specific preference distributions $\mathcal{D}_1, \dots, \mathcal{D}_K$. The key steps of PA-RLHF are:

1. **Preference clustering:** Identify latent groups G_k using embedding-based clustering of annotator preference vectors or label co-occurrence patterns.
2. **Per-group reward modeling:** Fit a separate reward model r_{θ_k} for each group G_k using only that group's preference data.
3. **Fairness-weighted policy optimization:** Combine per-group reward signals during PPO training using a fairness weighting w_k that does not simply equal the group's population share $\alpha_k = |G_k|/|\mathcal{A}|$.

The fairness weighting w_k is the key design choice: uniform weighting ($w_k = 1/K$) maximizes procedural equity; intermediate weightings balance procedural and outcome fairness objectives.

Technical Formulation

Standard RLHF fits:

$$r_\theta = \arg \min_{\theta} \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [-\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))]$$

where $\mathcal{D} = \sum_k \alpha_k \mathcal{D}_k$ is the mixture distribution.

PA-RLHF fits K per-group reward models:

$$r_{\theta_k} = \arg \min_{\theta_k} \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_k} [-\log \sigma(r_{\theta_k}(x, y_w) - r_{\theta_k}(x, y_l))]$$

and optimizes the policy with the fairness-weighted reward:

$$r_{\text{fair}}(x, y) = \sum_{k=1}^K w_k \cdot r_{\theta_k}(x, y), \quad \sum_k w_k = 1$$

Procedural fairness is defined as: the gradient contribution of group G_k to the aggregate reward model should be proportional to its “representation right” rather than its population size. Formally, standard RLHF satisfies this only when $\alpha_k = 1/K$ (equal group sizes), which is rarely the case in practice.

Learning or Inference Procedure

PA-RLHF requires one additional step versus standard RLHF:

1. **Cluster annotators** into K groups (one-time offline step).
2. **Train K reward models** in parallel (same cost as standard training when using parameter-efficient methods; $K \times$ cost otherwise).
3. **Run PPO** with r_{fair} as the reward signal.

Inference is identical to standard RLHF — no per-user data is required.

What the Guarantee Says

The paper proves that standard RLHF is procedurally unfair whenever annotator group sizes differ ($\alpha_k \neq 1/K$) and group preferences are non-identical. PA-RLHF with uniform weighting ($w_k = 1/K$) is the unique solution that achieves procedural fairness under this definition. No guarantee is provided for outcome fairness — PA-RLHF may improve or maintain outcome metrics, but cannot guarantee it.

Experimental Findings

- **Procedural fairness:** PA-RLHF improves procedural fairness metrics across all tested scenarios with heterogeneous annotator groups.
- **Outcome quality:** Win rates and helpfulness metrics are maintained or slightly improved relative to the standard RLHF baseline.
- **Group imbalance sensitivity:** Gains grow with group size imbalance ratio — the regime most common in real crowdsourced annotation datasets.
- **Preference divergence sensitivity:** Gains also grow with inter-group preference divergence, confirming the mechanism targets the right failure mode.

Ablations and Interpretation

How many groups K ? The paper tests $K \in \{2, 3, 5\}$; fairness gains are present for all values, with $K = 3$ providing the best balance of fairness and computation overhead.

Fairness weighting w_k : Uniform ($w_k = 1/K$) maximizes procedural fairness but can hurt outcome quality in extreme scenarios. Population-proportional ($w_k = \alpha_k$) recovers standard RLHF. An intermediate value provides the best empirical outcome-fairness tradeoff.

What about outcome fairness? The paper notes that procedural fairness and outcome fairness are not equivalent: PA-RLHF improves procedural fairness without guaranteeing outcome fairness, and a system can achieve outcome fairness while violating procedural fairness. Both matter and both need to be measured.

Reference

M P V S Gopinadh, Karthik Kamuju, Kummari Avinash, John Joshua, Srinivasa Raju Rudraraju. **Procedural Fairness Failures in RLHF from Preference Averaging**. arXiv:2608.10126, August 2026. ICLR 2026 Workshop on Algorithmic Fairness Across Alignment Procedures and Agentic Systems. <https://arxiv.org/abs/2608.10126>

Keywords: robotic manipulation • sim-to-real transfer • synthetic data • generalization benchmark

RoboSynChallenge: Mastering Real-World Dexterity via Generalizing Synthesized Manipulation Skills

NeurIPS 2026 Competition Pairs Synthetic-Only Training with Unseen Real-World Evaluation to Benchmark Generalizable Robot Policies at Scale

By Runyi Zhao, Ruixin Wu, Chengkun Li, Hongrui Zhang, Ang Li, Ruixing Jin, Yueci Deng, Yingying Guo, et al.

arXiv:2608.12416 • NeurIPS 2026 Competition Track

Generalizable robotic manipulation remains blocked by the scarcity and narrow diversity of real-world demonstration data. RoboSynChallenge (NeurIPS 2026 Competition Track) introduces a unified benchmark pairing large-scale synthetic data generation with standardized real-world evaluation on unseen environments: participants train policies from synthesized state-action trials, and final assessment occurs exclusively on real manipulation tasks they have never seen.

AT A GLANCE

Problem: Generalizable robot manipulation policies are bottlenecked by scarce, narrow real-world data; synthetic data is abundant but sim-to-real transfer is unreliable.

Why it matters: Standardized evaluation of sim-to-real generalization at scale does not exist; RoboSynChallenge provides it for the first time at NeurIPS scope.

Method: Participants train on a large synthetic state-action corpus; evaluation is conducted exclusively on unseen real-world manipulation environments.

Result: The competition provides a three-tier difficulty benchmark (basic, dexterous, open-vocabulary) and cross-institutional comparison of policy approaches.

Evidence: NeurIPS 2026 Competition Track participation ensures broad community coverage spanning diffusion policies, flow matching, and VLA architectures.

The Big Picture

The robotics community has long believed that data scarcity is the core obstacle to generalizable manipulation: if we had enough diverse demonstrations, robot policies would generalize the way vision models did after

ImageNet. Synthetic data is the obvious supply-side fix — simulators are cheap, parallelizable, and can generate arbitrary diversity of objects, grasp configurations, and physics parameters.

The catch is sim-to-real transfer. Policies trained on synthetic data encounter the real world and fail: the physics is subtly wrong, the visual domain is shifted, the robot’s dynamics differ from the simulated model. Decades of research have produced methods for closing the sim-to-real gap — domain randomization, system identification, residual dynamics learning — but evaluating how well any of these methods generalizes across tasks has been impossible without a standardized benchmark.

RoboSynChallenge is that benchmark.

Why Existing Approaches Fall Short

Existing robotics benchmarks suffer from one of two failure modes:

Real-data benchmarks (e.g., RoboMimic, Language-Table) evaluate in-distribution generalization: test tasks are drawn from the same distribution as training demonstrations. They measure *learning efficiency*, not *generalization beyond the training distribution*.

Simulation benchmarks (e.g., RLBench, MetaWorld) remove the sim-to-real problem by evaluating inside the simulator. They measure *task variety* but not the critical question: does this work on a real robot in the real world?

RoboSynChallenge is the first benchmark to pair synthetic training with real-world evaluation at the scale of a major venue competition, providing a clean measurement of the full sim-to-real generalization pipeline.

Core Mechanism

The challenge is structured as a two-phase competition:

Phase 1 (Synthetic Training): Participants receive a large-scale corpus of (s_t, a_t) state-action pairs generated in simulation across diverse object categories, task types, and environment configurations. Tasks include:

- Pick-and-place with novel objects and arrangements
- Articulated object manipulation (cabinets, bottles, drawers)
- Tool use requiring contact-rich dexterous control

Participants may augment with additional synthetic data using provided simulators.

Phase 2 (Real-World Evaluation): Trained policies are deployed on a standardized physical robot platform. All evaluation tasks are unseen — new objects, new goal configurations, new environments. Evaluation is fully standardized across participants: identical robot hardware, identical evaluation protocol, identical success metrics.

Technical Formulation

A manipulation policy $\pi : \mathcal{O} \times \mathcal{G} \rightarrow \mathcal{A}$ maps observations $o \in \mathcal{O}$ and goal specifications $g \in \mathcal{G}$ to actions $a \in \mathcal{A}$. The challenge tests π on the distribution shift:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{(o,g) \sim \mathcal{D}_{\text{real}}} [\text{Succ}(\pi, o, g)]$$

where $\mathcal{D}_{\text{real}}$ is the real-world evaluation distribution, but training data comes exclusively from $\mathcal{D}_{\text{syn}}$ (the synthetic corpus).

The generalization gap is defined as:

$$\Delta_{\text{gen}} = \mathbb{E}_{\mathcal{D}_{\text{syn}}} [\text{Succ}] - \mathbb{E}_{\mathcal{D}_{\text{real}}} [\text{Succ}]$$

Minimizing Δ_{gen} is the core objective.

Three evaluation tiers reflect difficulty:

Tier 1: $\mathcal{T}_1$ (basic), Tier 2: $\mathcal{T}_2$ (dexterous), Tier 3: $\mathcal{T}_3$ (open-vocabulary)

with $\text{Succ}(\mathcal{T}_3) \leq \text{Succ}(\mathcal{T}_2) \leq \text{Succ}(\mathcal{T}_1)$ for all known baselines.

Learning or Inference Procedure

The competition is open to any policy architecture and training procedure, subject to the constraint of synthetic-only training. Expected approaches include:

- **Diffusion policies:** Generate action trajectories via score matching
- **Flow matching policies:** Conditional flow matching for action prediction
- **Vision-Language-Action (VLA) models:** Combine foundation model representations with action prediction heads
- **Domain randomization:** Randomize simulator physics/appearance to cover the real-world distribution

What the Guarantee Says

The challenge provides no formal guarantee about the generalizability of any policy architecture. It provides the *measurement infrastructure* to empirically determine which synthetic data generation strategies, policy architectures, and transfer techniques generalize best to unseen real-world tasks. The benchmark acts as a scoreboard for the field’s progress on this open problem.

Experimental Findings

As a competition paper, RoboSynChallenge reports baseline results rather than final participant results:

- **Tier 1 (basic manipulation):** Strong baselines achieve >60% success rate with domain randomization; simple pick-and-place transfers well.

- **Tier 2 (dexterous tasks):** Success rates drop to 20–35% for all baselines; contact-rich manipulation is the primary failure mode.
- **Tier 3 (open-vocabulary):** Below 10% success for all baselines, establishing a challenging frontier for VLA and instruction-following approaches.
- **Generalization gap:** $\Delta_{\text{gen}} > 40\%$ for all baselines, confirming substantial sim-to-real transfer challenge.

Ablations and Interpretation

Data diversity vs. volume: Preliminary analysis suggests diversity of object categories matters more than raw data volume for Tier 2–3 transfer, consistent with findings in the vision literature.

Visual vs. proprioceptive sim-to-real: Visual domain gap accounts for ~60% of the generalization gap for Tier 1 tasks; proprioceptive and contact dynamics gaps dominate for Tier 2–3.

Action representation: Policies trained with relative action targets transfer better than absolute-coordinate policies, suggesting coordinate-space choice is a significant source of sim-to-real failure.

Reference

Runyi Zhao, Ruixin Wu, Chengkun Li, Hongrui Zhang, Ang Li, Ruixing Jin, Yueci Deng, Yingying Guo, Lihe Ding, Shaocong Dong, Tianfan Xue, Yanjun Gao, Yudong Luo, Pascal Poupart, Simo Wu, Kui Jia, Wei-shi Zheng, Guiliang Liu. **RoboSynChallenge: Mastering Real-World Dexterity via Generalizing Synthesized Manipulation Skills**. arXiv:2608.12416, August 2026. NeurIPS 2026 Competition Track. <https://arxiv.org/abs/2608.12416>

Keywords: hallucination detection • retrieval-augmented generation • LLM reliability • uncertainty estimation

SIRIN: A Unified Toolkit for Detecting Contextual Hallucinations in RAG and Memory-Grounded LLM Systems

Three Detector Paradigms — Representation Probing, Uncertainty Estimation, and Judge-Style Verification — Unified Under One Interface and Evaluation Pipeline

By Julia Belikova, Rauf Parchiev, Mikhail Filimonov, Konstantin Polev, Andrey Savchenko, Maksim Makarenko

arXiv:2608.00033 • Preprint, August 2026

Contextual hallucinations — fluent, plausible LLM responses that contradict the provided retrieved evidence — are the dominant reliability failure in RAG systems. SIRIN (Semantic Inconsistency Recognition and Inspection Nexus) consolidates three detector paradigms under one unified toolkit, configuration system, and evaluation pipeline, supporting both white-box and black-box settings as well as pre-generation query answerability checks.

AT A GLANCE

Problem: Contextual hallucinations in RAG systems are invisible to standard quality metrics and require comparing model output to retrieved context.

Why it matters: RAG is used in high-stakes applications; hallucination detection is safety-critical but existing methods are fragmented across incompatible systems.

Method: Unify representation probing, uncertainty estimation, and judge-style verification under one interface; add pre-generation query answerability.

Result: No single paradigm dominates; combining uncertainty + judge-style verification improves precision; answerability filtering reduces hallucination rate.

Evidence: Benchmarked across standard RAG hallucination datasets in both white-box and black-box settings.

The Big Picture

Retrieval-augmented generation (RAG) addresses parametric hallucination by grounding LLM outputs in retrieved documents. But retrieval-grounded generation introduces its own failure mode: **contextual hallucination**, where the model generates a response that is fluent and internally coherent but contradicts or is unsupported

by the specific retrieved evidence. Unlike parametric hallucination (confabulated facts), contextual hallucination requires detecting inconsistency between model output and the provided context — a structured grounding task.

The detection literature is fragmented. Representation probing requires model internals. Uncertainty estimation can work in grey-box settings. Judge-style verification works via API but is expensive. No unified system allows fair comparison across paradigms on the same inputs with the same evaluation protocol — until SIRIN.

Why Existing Approaches Fall Short

Representation probing methods train a lightweight classifier on the model’s internal activations to predict hallucination. They are fast and model-specific but require white-box access and labeled probe training data.

Uncertainty estimation methods measure token-level or span-level prediction entropy as a proxy for hallucination risk. They are model-agnostic in principle but require access to logit distributions (grey-box) and can confuse *uncertain* content with *wrong* content.

Judge-style verification uses a second LLM to verify each claim in the response against the retrieved context. It is the most model-agnostic and robust but requires additional LLM calls (cost) and is sensitive to judge model quality.

Each paradigm has a different interface, codebase, and evaluation protocol, making cross-comparison impossible without SIRIN’s unification layer.

Core Mechanism

SIRIN provides three core components:

1. Detector modules implementing each paradigm:

- *Probe detector:* Trains a linear probe on activations from a specified layer to classify hallucinated vs. grounded spans.
- *Uncertainty detector:* Computes semantic entropy, predictive entropy, or contrastive uncertainty over the model’s token distribution.
- *Judge detector:* Prompts a judge LLM to verify each factual claim in the response against the retrieved context chunk-by-chunk.

2. Answerability module: Before generation, scores whether the retrieved context contains sufficient information to answer the query. If answerability is below threshold, the system abstains rather than generates a potentially hallucinated response.

3. Unified evaluation pipeline: Standard datasets (TriviaQA-RAG, NQ-RAG, HaluEval-RAG), shared metrics (precision, recall, F1 at response and span level), and a comparison framework that runs multiple detectors on

the same inputs.

Technical Formulation

Let c be the retrieved context, q the query, and y the generated response. Contextual hallucination detection is a binary classification:

$$d(y, c) = \begin{cases} 1 & \text{if } y \text{ is inconsistent with } c \\ 0 & \text{otherwise} \end{cases}$$

Probe detector: Train $f_\phi : \mathbf{h} \rightarrow [0, 1]$ where $\mathbf{h} = \text{enc}_\theta(y, c)$ is the model’s internal representation:

$$d_{\text{probe}}(y, c) = \mathbb{K}[f_\phi(\mathbf{h}) > \tau_p]$$

Uncertainty detector: Estimate semantic entropy as:

$$H_{\text{sem}} = - \sum_{s \in S} p(s \mid c, q) \log p(s \mid c, q)$$

where S is the set of semantically distinct responses sampled via beam search or Monte Carlo sampling. High entropy signals hallucination risk.

Judge detector: For each extracted claim $\hat{y}_i$:

$$d_{\text{judge}}(\hat{y}_i, c) = \text{LLM}_{\text{judge}}(\text{“Is } \hat{y}_i \text{ supported by } c\text{?”})$$

Answerability: Pre-generation query answerability score:

$$a(q, c) = \text{LLM}_{\text{judge}}(\text{“Does } c \text{ contain enough info to answer } q\text{?”})$$

Learning or Inference Procedure

White-box setting: Probe detectors require one-time fine-tuning on a labeled hallucination dataset (~1k examples). Uncertainty detectors require multiple forward passes (5–20 samples) per query. No training for the judge detector.

Black-box setting: Only the judge detector is applicable. SIRIN wraps the judge calls with batching and rate limiting for production deployment.

What the Guarantee Says

No single detector paradigm dominates across all settings. The key empirical guarantee is: combining uncertainty estimation + judge-style verification achieves higher precision than either alone, at moderate additional cost. Answerability filtering reduces hallucination rate by 15–25% by preventing generation when context is insufficient.

Experimental Findings

- **Probe detector:** Best F1 in white-box settings; poor in black-box.

- **Uncertainty detector:** Competitive in grey-box; highest recall but lower precision than judge-style.
- **Judge detector:** Most consistent across settings; best in black-box.
- **Combination:** Uncertainty + judge achieves the best precision-recall tradeoff across all datasets.
- **Answerability filtering:** Reduces hallucination rate 15–25% by abstaining when context is insufficient.

Ablations and Interpretation

Span vs. response level: Span-level detection has higher recall but lower precision than response-level detection; the interactive UI benefits from span-level highlighting for analyst workflows.

Judge model quality: Detection performance degrades gracefully as judge model size decreases; 7B judge models achieve >85% of the performance of frontier judges at 10× lower cost.

Answerability threshold: High thresholds ($a > 0.9$) maximize precision at the cost of high abstention rate; $a > 0.7$ is the practical operating point.

Reference

Julia Belikova, Rauf Parchiev, Mikhail Filimonov, Konstantin Polev, Andrey Savchenko, Maksim Makarenko. **SIRIN: A Unified Toolkit for Detecting Contextual Hallucinations in Retrieval-Augmented and Memory-Grounded LLM Systems.** arXiv:2608.00033, August 2026. <https://arxiv.org/abs/2608.00033>